P3474R0

`std::arguments`

Published Proposal, 2024-10-15

This version:
https://isocpp.org/files/papers/P3474R0.html

Author:
Jeremy Rifkin

Audience:
SG17, SG18

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:
https://github.com/jeremy-rifkin/proposals/blob/main/cpp/arguments.bs

---

Abstract

This paper proposes an encoding-friendly and modern interface for accessing command line arguments throughout a program.

Table of Contents

1 Credits

2 Introduction

3 Previous Straw Polls and Discussion

4 Implementability

5 Proposed Design

5.1 Design Considerations

5.2 Future Interface Expansion
5.3 Bikeshedding

6 Reference Implementation

7 Proposed Wording

References
Normative References
Informative References

§ 1. Credits

`std::arguments` was initially proposed by Izzy Muerte in [P1275]. Corentin Jabot and Aaron Ballman also proposed an interface for accessing command line arguments outside `main` to WG14 in [N2948]. This paper borrows wording, design elements, and good ideas from both.

§ 2. Introduction

This paper aims to solve three problems: Encoding and portability problems with command line arguments, an interface for accessing arguments outside of main, and a modern interface for accessing arguments. It does so by introducing a `std::arguments` class that provides a modern and encoding-friendly interface for arguments.

Encoding: The only standard means for accessing access command-line arguments in C++ is via `int main(int argc, char** argv)`. This is a staple of C and C++, however, it's not well-suited for portable applications because the encoding of `argv` varies system to system [[What is the encoding of argv?]]. On Windows, the native encoding is UTF-16 and it's recommended to use `wmain` instead of `main` for portable code. In order to facilitate `argv`, UTF-16 arguments must be converted using legacy Windows code pages. The preferred ways to handle command line arguments on Windows are platform-specific functions, `WinMain`, or `wmain`. Even on Unix-based systems the encoding of `char** argv` is not always clear. Tackling this problem more or less necessitates an interface for accessing command line arguments independent of `main` as adding a new signature to `main` has been rejected by the committee.

Access outside main: It may be desirable to access command line arguments outside of `main` and even to do so before `main`. Some examples could include logging diagnostic information in a crash handler and some designs for a command line argument parser. Currently, command line arguments

are only available inside of `main` which requires a programmer to manually pass arguments throughout the program or create their own global storage for arguments. This can add clutter and introduce unnecessary complexity, especially if argument handling doesn't happen "close" to `main`. There is precedent from other languages for global access, notably languages such as Python, Go, Rust, Swift, Ruby, C#, Haskell, Ada, and many others provide an interface for accessing arguments from anywhere in a program. Additionally, many C++ frameworks make arguments available outside `main`, such as QT with `QCoreApplication::arguments`.

Modernity: Passing arrays via a pointer and length argument is a very antiquated pattern rendered obsolete by modern solutions such as `std::span`. `main` is the one case where separate pointer and length arguments are still a requirement if command line arguments are desired. A modern signature for `main` along the lines of `int main(std::span<char*> argv)`, `int main(std::span<std::string_view> argv)`, or `int main(std::argument_list argv)` was previously rejected by the committee due to concerns surrounding complexity, overhead, and encoding issues [P0781]. On top of new functionality and increased portability, a facility such as `std::arguments` provides a modern C++ solution for accessing arguments. An important benefit to this interface is teachability: Currently command line arguments require introduction to pointers relatively early on in education as well as subjection to footguns and confusion about the difference between C strings and C++ strings. This adds steepness to an already hazardously steep learning curve.

§ 3. Previous Straw Polls and Discussion

Early polling surrounding an alternative to `argc`/`argv` and a means of accessing arguments outside of `main` occurred during discussion of [P0781]:

> POLL: A trivial library solution for iterating parameters?
>
SF	F	N	A	SA
2	12	14	2	1

> POLL: A non-`main`-based way of fetching command line arguments?
>
SF	F	N	A	SA
7	9	9	1	2

Polls on [P1275] by LEWGI:

> POLL: We should promise more committee time to the `std::arguments` part.
> Unanimous consent
> Attendance: 11

> POLL: `std::arguments` should be available before main
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 6 | 0 | 3 | 1 | 0 |
>
> Attendance: 11

Polls on [P1275] by SG16:

> POLL: `std::environments` and `std::arguments` should follow the precedent set by `std::filesystem::path`.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 4 | 6 | 1 | 0 | 2 |
>
> Attendance: 14

> POLL: `std::environment` and `std::arguments` should return a bag-o-bytes and conversion is up to the user.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 3 | 4 | 2 | 1 | 2 |
>
> Attendance: 14

Key concerns discussed included mutability of arguments, overhead of initializing data structures before `main`, and how to handle different encodings.

§ 4. Implementability

On Windows, command line arguments can be accessed by `GetCommandLineW`. This function returns the command line as a string which must then be tokenized. This is called by the Windows CRT during startup to populate `argv` for main. The Windows CRT also provide `__argv` and `__wargv` global variables but only populates one depending on `__UNICODE__`. Additionally, neither may be populated if the command line parsing is disabled via options tailored to applications trying to minimize startup time.

On MacOS, `_NSGetArgv` and `_NSGetArgc` can be used to access `argc` and `argv` outside of `main`. These are both trivial functions that don't allocate.

Implementation on other Unix-based systems is more challenging. There are four options:

1. Modify libc to store `argv` and `argc` globally, e.g. `__argc` and `__argv`, similar to `__environ`. (reference implementation for this from N2948).
2. Alternatively, store `argc` and `argv` from the program's entry point. This would only require compiler support instead of a libc change.
3. Use `__dl_argv` which exists in glibc. Unfortunately, absent a glibc change, looping through `__dl_argv` would be needed to determine `argc` as `__dl_argc` is hidden.
4. Read from and tokenize `/proc/self/cmdline`.
5. Glibc passes `argc` and `argv` to entries in the `.init_array`.

§ 5. Proposed Design

This paper introduces two classes, `std::arguments` and `std::argument`, and a header, `<arguments>`.

`std::arguments` has the interface of a constant `std::span` excluding the subview interface, modifiers, constructors, `size_bytes`, and `data`. Its default constructor initializes the object to represent the program's command-line arguments and may perform allocation.

`std::arguments` has a `value_type` of `std::argument` which mirrors the design of `std::filesystem::path` by providing observers that can convert to desired encodings. SG16 indicated a desire to follow the precedent of `std::filesystem::path`. Both paths and arguments can be encoded arbitrarily or even have no encoding; paths could be any sequence of bytes and command line arguments can be too. `std::argument` may be a view of a string or may own an allocation.

While it is not uncommon practice to modify the contents of `char** argv`, `std::arguments` is entirely read-only in order to not introduce dangers surrounding global mutable state. Whether changes made to `argv` in `main` are reflected in `std::arguments` is implementation-defined.

§ 5.1. Design Considerations

The main design considerations come down to allocation, when tokenization or other argument preprocessing happens, and whether modifications to `argv` in `main` are reflected in `std::arguments`.

Reflecting `argv` modifications from `main`: It is desirable for `std::arguments` to contain the same values throughout the lifetime of a program and to not reflect changes to `argv` in `main`. Unfortunately, this would require allocation and copying on some systems. On Unix-based systems all means to access `argv` will reflect changes to `argv` in `main`, including `/proc/self/cmdline`. Discussion on [P1275] and [P0781] made clear that any overhead before `main` in the case of programs that don't use `std::arguments` is unacceptable. Unfortunately, an initializer similar to `std::ios_base::Init` isn't an option due to shared libraries not necessarily being loaded before `main`. Additionally, with `import std;` this would translate to overhead before `main` that is not pay for what you use. Due to implementations challenges, this paper leaves behavior implementation-defined in the case of `argv` being modified in `main`.

Saving `strlen`: On Unix-based systems, producing string views for arguments will involve a `strlen`. It may be desirable to save the result of this computation, however, the issue of modification mostly rules this out. While the storage for the arguments from the system will always be there, the pointers in `argv` could be modified and detecting this would be sufficiently complicated, involve overhead, or in general may be impossible. Because of this, every access of an argument string view will require a `strlen` unless the implementation makes copies of `argv` string entries. It would likely be undesirable to make it undefined behavior to use `std::arguments` after modifications in `main` so this paper leaves the possibility of a strlen cost open.

Preprocessing: On Windows `GetCommandLineW` will return a string which needs to be split into individual arguments. It may be desirable in some use-cases to only split this string lazily with an input-iterator interface for arguments. This paper does not suggest any design constrained to input-iteration, though, as much use will want more general access and iteration abilities and will require having tokenized all arguments anyway - whether by looping through all the arguments or even just looking at the argument count.

Backing storage for `std::argument`s: On Unix-based systems it would be simple for `std::arguments` to not involve any allocation and simply provide iterators over `argv` that dereference to ephemeral `std::argument` objects. Unfortunately, this would prevent the iterator from satisfying the *Cpp17RandomAccessIterator* requirements, container requirements, and may be error prone in the case of trying to store a reference to a `std::argument`. The proposed requirements here will require backing storage.

Global singleton, a function returning a reference, or construction: `std::arguments` could be implemented as a global singleton similar to `std::cout`, a `std::arguments` function returning a reference to a singleton, or as an object that the user constructs. While an object the user constructs potentially results in allocation at multiple points in a program, as well as possibly seeing different values if `argv` is modified in `main`, it's also desirable to allow the `std::arguments` allocation to

be cleaned up. As such, this paper proposes a `std::arguments` class which may perform allocation and various preprocessing at construction.

Globs and `argv[0]`: On Unix-based systems glob expansion is done by the shell. On Windows it is neither done by the shell or the Windows CRT. This paper proposes `std::arguments` should correspond directly to `argv` in `main` without any additional glob expansion. This paper also does not propose any special handling for the first entry of `argv`.

Comparison with other performance-oriented languages: Rust's `std::env::args()` function creates an `Args` object which involves creating a vector of strings in the OS native encoding, copying from `argv` on Unix-based systems and tokenizing on Windows. Rust accesses `argv` and `argc` on most Unix-based systems by placing an initializer in the `.init_array`. Rust doesn't have to worry about modification of `argv` in `main`.

Because the design of this library feature involves a lot of tradeoffs, it is the goal of this paper to offer as much implementation flexibility as possible.

§ 5.2. Future Interface Expansion

Author's note: While most large applications should probably use a library for argument parsing, it is my hope that in the case of more ad-hoc argument parsing it would be possible to portably write a check such as `std::arguments.at(1) == "--help"` or `std::arguments.at(1).native() == "--help"`. Another helpful operation would be `.starts_with("--")`. Unfortunately, encoding makes it challenging to do operations such as this portably.

Because encoding will vary between systems and `native()` is implementation-defined, currently the only way to do this would involve the overhead of creating a string for a given encoding or an ugly macro to create a platform-dependent string literal:

```
// The overhead here is unfortunate but OK for 99% of uses
if(std::arguments.at(1).string() == "--help") {
  // ...
}

// or:

#ifdef _WIN32
#define ARG(str) L##str
#else
#define ARG(str) str
```

```
#endif
if(std::arguments.at(1).native() == ARG("--help")) {
  // ...
}
```

A UDL could also be considered, however, this is a more general problem that, in the author's opinion, should be addressed directly rather than through a bespoke solution. The problem of operations between strings of different encodings would best be tackled in another paper.

§ 5.3. Bikeshedding

This paper uses the `std::arguments` naming from [P1275], however, the name is subject to bikeshedding. One point brought up on the mailing list was that `arguments` is a very generic name and it might be desirable to reserve it for future use. Some names that could be considered instead include:

- `std::program_arguments`
- `std::command_line`
- `std::command_line::arguments`
- `std::program_options`
- `std::argv`
- `std::process:arguments`

Naming in other notable languages:

- Python: `sys.argv`
- Go: `os.Args()`
- Rust `std::env::args()`
- Swift: `CommandLine.arguments`
- Ruby: `ARGV`
- C#: `Environment.GetCommandLineArgs()`
- Haskell: `getArgs()`
- Ada: `Ada.Command_Line.Argument`

In a very informal approval-voting-style poll on the Together C & C++ Discord server (participants were asked to vote for all they found appealing) members showed a strong preference for either `std::arguments` or `std::argv` with eight and 17 votes respectively. Other options had no more than two votes. N.b.: The last option, `std::process::arguments`, came up after the poll was started and thus wasn't captured in the poll.

6. Reference Implementation

A reference implementation / proof of concept is at https://github.com/jeremy-rifkin/arguments.

7. Proposed Wording

Wording is relative to [N4950] and borrows extensively from existing wording.

Insert into [headers] table 24:

> `<arguments>`

Insert a new section [arguments]:

Header `<arguments>` synopsis [arguments.syn]

```
namespace std {
  class argument;
  template<class Allocator = allocator<argument>> class arguments;
}
```

Class `arguments` [arguments.view]

Class `arguments` is a read-only container holding a continuous range of `argument` objects corresponding to arguments passed to the program.

All member functions of `arguments` have constant time complexity except the constructor.

```
namespace std {
  template<class Allocator = allocator<argument>>
  class arguments {
  public:
    using value_type = const argument;
    using size_type = size_t;
    using difference_type = ptrdiff_t;
    using pointer = value_type*;
    using const_pointer = value_type*;
    using reference = value_type&;
    using const_reference = value_type&;
    using const_iterator = /* implementation-defined */; // see [argume
    using iterator = const_iterator;
    using const_reverse_iterator = std::reverse_iterator<const_iterator
    using reverse_iterator = const_reverse_iterator;

    // [arguments.view.cons], constructors
    arguments() noexcept(noexcept(Allocator())) : arguments(Allocator()
    explicit arguments(const Allocator&);

    // [arguments.view.access], access
    reference operator[](size_type index) const noexcept;
    reference at(size_type index) const;

    // [arguments.view.obs], observers
    size_type size() const noexcept;
    bool empty() const noexcept;

    // [arguments.view.iterators], iterators
    const_iterator begin() const noexcept;
    const_iterator end() const noexcept;

    const_iterator cbegin() const noexcept;
    const_iterator cend() const noexcept;

    const_reverse_iterator rbegin() const noexcept;
    const_reverse_iterator rend() const noexcept;

    const_reverse_iterator crbegin() const noexcept;
```

```
    const_reverse_iterator crend() const noexcept;
  };
}
```

§ Constructors [arguments.view.cons]

`explicit arguments(const Allocator&) noexcept;`

Effects: Constructs an `arguments` object with the program's arguments using the specified allocator.

Throws: May throw if `Allocator::allocate` throws.

§ Access [arguments.view.access]

`value_type operator[](size_type index) const noexcept;`

Preconditions: `index < size()` is `true`.

Returns: The argument at index `index` passed into the program from the environment. It is implementation-defined whether, in a `main` function with signature `main(int argc, char** argv)`, any modifications to `argv` are reflected by `arguments::operator[]`.

Throws: Nothing.

[Note 1: `operator[](index)` corresponds to `argv[index]` in `main(int argc, char** argv)` — end note].

`value_type at(size_type index) const;`

Effects: Equivalent to: `return operator[](index);` if `index >= size()` is `true`.

Throws: `out_of_range` if `index >= size()` is `true`.

§ Observers [arguments.view.obs]

`size_type size() const noexcept;`

Returns: The number of program argument.

```
size_type empty() const noexcept;
```

Effects: Equivalent to: `return size() == 0;`

Iterators [arguments.view.iterators]

```
using const_iterator = /* implementation-defined */;
```

The type models a `contiguous_iterator` ([iterator.concept.contiguous]) and meets the *Cpp17RandomAccessIterator* requirements ([random.access.iterators]) whose value type is `value_type` and whose reference type is `reference`.

All requirements on container iterators ([container.reqmts]) apply to `arguments::iterator` as well.

```
const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;
```

Returns: An iterator referring to the first program argument. If `empty()` is `true`, then it returns the same value as `end()`.

```
const_iterator end() const noexcept;
const_iterator cend() const noexcept;
```

Returns: An iterator which is the past-the-end value.

```
const_iterator rbegin() const noexcept;
const_iterator crbegin() const noexcept;
```

Effects: Equivalent to: `return reverse_iterator(end());`

```
const_iterator rend() const noexcept;
const_iterator crend() const noexcept;
```

Effects: Equivalent to: `return reverse_iterator(begin());`

Class `argument` [arguments.argument]

An object of class `argument` is a view of a character string argument passed to the program in an operating system-dependent format.

It is implementation-defined whether, in a `main` function with signature `main(int argc, char** argv)`, any modifications to `argv` are reflected by an `argument`.

```
namespace std {
  class argument {
  public:
    using value_type  = /* see below */;
    using string_type = basic_string<value_type>;
    using string_view_type = basic_string_view<value_type>;

    // [arguments.argument.native], native observers
    const string_view_type native() const noexcept;
    const string_type      native_string() const;
    const value_type*      c_str() const noexcept;
    explicit operator string_type() const;
    explicit operator string_view_type() const noexcept;

    // [arguments.argument.obs], converting observers
    template<class EcharT, class traits = char_traits<EcharT>,
             class Allocator = allocator<EcharT>>
      basic_string<EcharT, traits, Allocator>
        string(const Allocator& a = Allocator()) const;
    std::string    string() const;
    std::wstring   wstring() const;
    std::u8string  u8string() const;
    std::u16string u16string() const;
    std::u32string u32string() const;

    // [arguments.argument.compare], comparison
    friend bool operator==(const argument& lhs, const argument& rhs) no
    friend strong_ordering operator<=>(const argument& lhs, const argum

    // [arguments.argument.ins], inserter
    template<class charT, class traits>
      friend basic_ostream<charT, traits>&
        operator<<(basic_ostream<charT, traits>& os, const argument& a)
  };

  // [arguments.argument.fmt], formatter
  template<typename charT>
    struct formatter<argument, charT>
      : formatter<argument::string_view_type, charT> {
```

```
        template<class FormatContext>
          typename FormatContext::iterator
            format(const argument& argument, FormatContext& ctx) const;
    };
}
```

Conversion [arguments.argument.cvt]

The *native encoding* of an ordinary character string is the operating system dependent current encoding for arguments. The *native encoding* for wide character strings is the implementation-defined execution wide-character set encoding ([character.seq]).

For member functions returning strings, value type and encoding conversion is performed if the value type of the argument or return value differs from `argument::value_type`. For the return value, the method of conversion and the encoding to be converted to is determined by its value type:

- `char`: The encoding is the native ordinary encoding. The method of conversion, if any, is operating system dependent.
- `wchar_t`: The encoding is the native wide encoding. The method of conversion is unspecified.
- `char8_t`: The encoding is UTF-8. The method of conversion is unspecified.
- `char16_t`: The encoding is UTF-16. The method of conversion is unspecified.
- `char32_t`: The encoding is UTF-32. The method of conversion is unspecified.

If the encoding being converted to has no representation for source characters, the resulting converted characters, if any, are unspecified.

Native Observers [arguments.argument.native]

The string returned by all native observers is in the native default argument encoding ([arguments.argument.cvt]).

`const string_view_type native() const noexcept;`

Returns: A `string_view_type` representing the argument.

`const string_type native_string() const;`

Returns: A `string_type` representing the argument.

```
const value_type* c_str() const noexcept;
```

Returns: A pointer to a null-terminated array of `value_type` representing the argument.

```
operator string_type() const;
```

Returns: A `string_type` representing the argument.

```
operator string_view_type() const noexcept;
```

Returns: A `string_type` representing the argument.

§ Converting Observers [arguments.argument.obs]

```
template<class EcharT, class traits = char_traits<EcharT>,
          class Allocator = allocator<EcharT>>
  basic_string<EcharT, traits, Allocator>
    string(const Allocator& a = Allocator()) const;
```

Returns: A string representing the argument.

Remarks: All memory allocation, including for the return value, shall be performed by a. Conversion, if any, is specified by [arguments.argument.cvt].

```
std::string string() const;
std::wstring wstring() const;
std::u8string u8string() const;
std::u16string u16string() const;
std::u32string u32string() const;
```

Returns: A string representing the argument.

Remarks: Conversion, if any, is specified by [arguments.argument.cvt].

§ Comparison [arguments.view.compare]

```
friend bool operator==(const argument& lhs, const argument& rhs) noexcept;
```

Effects: Equivalent to: `return lhs.native() == rhs.native();`.

```
friend strong_ordering operator<=>(const argument& lhs, const argument&
rhs) noexcept;
```

Effects: Equivalent to: `return lhs.native() <=> rhs.native();`.

§ Inserter [arguments.argument.ins]

```
template<class charT, class traits>
  friend basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const argument& a);
```

Effects: Equivalent to: `return os << a.string<charT, traits>();`.

§ Formatter [arguments.argument.fmt]

```
template<class FormatContext>
  typename FormatContext::iterator
    format(const argument& argument, FormatContext& ctx) const;
```

Effects: Equivalent to: `return std::formatter<argument::string_view_type>::format(argument.string<charT, char_traits<charT>>(), ctx);`.

§ References

§ Normative References

[N4950]
Thomas Köppe. *Working Draft, Standard for Programming Language C++*. 10 May 2023. URL: https://wg21.link/n4950

§ Informative References

[N2948]
Accessing the command line arguments outside of main(). URL: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2948.pdf

[P0781]

A Modern C++ Signature for main. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0781r0.html

[P1275]

Desert Sessions: Improving hostile environment interactions. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1275r0.html

[What is the encoding of argv?]

What is the encoding of argv?. URL: https://stackoverflow.com/questions/5408730/what-is-the-encoding-of-argv